

Time Complexity Done

List ADT

- When ADT mentioned no coding mention in sentences
- List refers to a collection of data items of same type
 - i) Groceries to be purchased
 - ii) Ingredients of a recipe
 - iii) To-do list
- Flexible Structure: Can grow or shrink on demand
- Elements can be inserted, Accessed, Deleted at any position/Index



→ Zero or more elements represented by a sequence of elements separated by comma: $a_1, a_2, \dots, a_n$

→ When $n \geq 0$, each a_i is of same data type

→ Number of elements n determines length of list

Properties of Lists

i) Can have a single element

ii) Can have no elements

iii) Can be list of lists

iv) Can be concatenated together

v) Can be split into sub-lists

List as an ADT

i) Homogenous

ii) Finite length

iii) Sequential elements

Basic Operations

i) Create the list: Initialized to empty state

ii) Determine whether list is empty

iii) Determine whether list is full

iv) Find the size of list

v) Destroy, or clear, the list

vi) Insert an item in the list at specified location

vii) Delete an item from the list at specified location

viii) Replace an item at specified location with another item

ix) Retrieve an item from list at the specified location

x) Search the list for a given item

xi) Transverse elements of the list

LUCKY

- Insert (x, p, L)
 - Insert at position p in list L
 - If list L has no position p result is undefined
- Retrieve (p, L)
 - Return element at position p on List L
- Locate (x, L)
 - Return the position of x on List L
- Delete (p, L)
 - Delete element at position p on List L
- MakeNull(L)
 - Causes L to become an empty list & returns position END(L)
- Next(p, L)
 - Return position following p on List L
- Previous (p, L)
 - Return position preceeding p on List L
- First (L)
 - Returns first position on List L
- PrintList (L)
 - Print elements of L in order of occurrence

List as a Data Structure

- We know ADT of list but how to implement it
- Create a List class at least the following function
 - i) is Empty() (ii) insert() (iii) delete() (iv) print() (v) is Full()
 - vi) list Size() (vii) retrieve() (viii) replace() (ix) search() (x) clear List()
- Implementation Involves
 - i) Defining data members
 - ii) Defining Function members from design phase

LUCKY



- In terms of implementation, there are two possible approaches
- Array based implementation of list
 - Linked list using pointers - based implementation of lists

Array Based Implementation

- An array is a viable choice for storing list elements

Dynamic Allocation of List Class

changes required

- Eliminate constant declaration for Capacity/size
- Data members to store capacity specified by client program

Problem: Array has fixed capacity

- Use DMA

Problem: Class bound to one-type at a time

- Use class Template

DSA class 7

What is a Linked List

- A linked list is a linear data structure in which elements (nodes) store its own data and the memory address/pointer of the next node. The nodes are chained together through these pointers.

Parts	Purpose
Data	Stores the actual value/element of the list
Next	Stores a pointer to the next node. If no next node then stores NULL

Why use a linked list & not an array

- Arrays have a fixed size decided at creation; linked list grows/shrinks dynamically one node at a time
- Inserting or deleting in the middle of an array requires shifting every subsequent element; a linked list only needs to rewire a couple of pointers
- Array can be accessed anywhere; Linked list has to be traversed one by one

Building Blocks

struct node {

int data;

node *next;

}

class List {

private:

node *head;

public:

list() { head = NULL; }

};

LUCKY



Operations:

- i) Is Empty: Is the list Empty ($head == NULL$)
- ii) Insert Node: insert a new node at a given position
- iii) Find Node: Search for a node with a given value
- iv) Delete Node: delete a node with a given value or position
- v) Display List: print every node's data from head to last node

Insert at start

```
void insert-at-start(int v) {
    node * temp = new node; // Allocates memory for new node on heap; temp points
    temp->data = v; // Stores value at data & next = NULL
    temp->next = NULL;
    if (head == NULL) { // New node becomes list so head = temp
        head = temp;
    } else { // new node becomes new front
        temp->next = head;
        head = temp;
    }
    temp = NULL; // Memory cleanup
    delete temp;
}
```



Mon Tue Wed Thu Fri Sat Sun

Date

```
Void insert_to_end (int v) {  
    node *temp = new node;  
    temp->data = v;  
    temp->next = NULL;  
    if (head == NULL) {  
        head = temp;  
    }  
    else {  
        node *p = head;  
        while (p->next != NULL) {  
            p = p->next;  
        }  
        p->next = temp;  
    }  
    temp = NULL;  
    delete temp;  
}
```

```
Void insert_at_location (int l, int v) {  
    node *temp = new node; temp->data = v; temp->next = NULL;  
    node *p = head; node *q = head->next;  
    for (int i = 1; i < l - 1; i++) {  
        p = q; q = q->next;  
    }  
    p->next = temp;  
    temp->next = q;  
}
```



```
Void Insert_After_Value (int c, int v) {  
    node *temp = new node; temp->data = v; temp->next = NULL; node *p = head;  
    while (p != NULL && p->data != c) {  
        p = p->next;  
    }  
    q = p->next; p->next = temp; temp->next = q; temp = NULL; delete temp;  
}
```

Deleting a Node

```
void delete_at_end() {  
    node *pre = head, *curr = head;  
    while (curr->next != NULL) {  
        pre = curr; curr = curr->next;  
    }  
    pre->next = NULL; delete curr; curr = NULL;  
}
```

```
void delete_at_start() {  
    node *q = head;  
    head = head->next;  
    delete q;  
}
```

```
void delete_a_location (int l) {
```

```
    node *pre = head, *curr = head;
```

```
    for (int i = 1; i < l; i++) {
```

```
        pre = curr; curr = curr->next;
```

```
    }
```

```
    pre->next = curr->next; delete curr; curr = NULL;
```

```
}
```

```
void delete_a_value (int v) {
```

```
    node *pre = head, *curr = head; while (curr->data != v && curr->next != NULL) {
```

```
        pre = curr; curr = curr->next; } pre->next = curr->next; delete curr;
```

```
    curr = NULL }
```

LUCKY



Searching a Node

```
void search (int v) {
```

```
    node *p = head; int c = 0;
```

```
    while (P != NULL) {
```

```
        c++;
```

```
        if (P->data == v)
```

```
            break;
```

```
        p = p->next;
```

```
    }
```

```
    if (P != NULL)
```

```
        cout << "data found at " << c << endl;
```

```
}
```

Printing a List

```
void display () {
```

```
    if (head == NULL)
```

```
        cout << "empty" << endl;
```

```
    node *p = head;
```

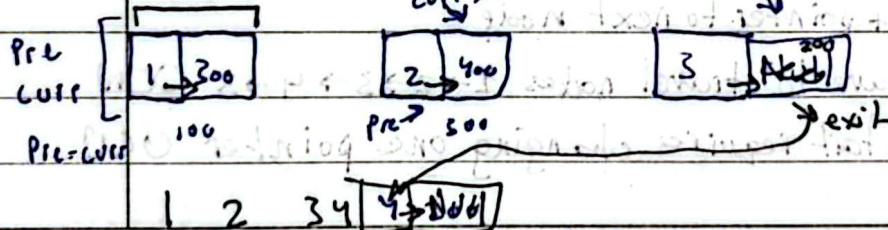
```
    while (p != NULL) {
```

```
        cout << p->data << " ";
```

```
        p = p->next;
```

```
}
```

```
*P
```



2/2

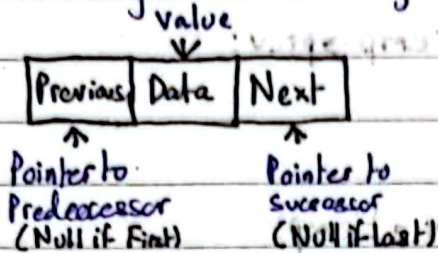


DSA class 9+10

• Doubly linked List

→ A singly linked list gives every node exactly one pointer to the next node.

→ The doubly linked list gives the second pointer that points backward



Struct Node {

Node * previous;

int data;

Node * next;

Class List {

private:

Node * head, * tail;

public:

List() {

head = NULL;

tail = NULL;

}



Insertion at head

void insert_at_head (int v) {

node *temp = new node; temp->data = v; temp->prev = NULL;

temp->next = NULL;

if (head == NULL & tail == NULL) {

head = temp->next; tail = temp->prev;

}

else {

temp->next = head; head->prev = temp;

head = temp; }

}

Insert at end

void insert_at_end (int v) {

node *temp = new node; temp->data = v; temp->prev = NULL; temp->next = NULL;

if (head == NULL & tail == NULL) {

head = temp; tail = temp;

}

else {

tail->next = temp; temp->prev = tail; tail = temp;

}

Insert at a Specified Position

void insert_at_pos (int loc, int v) {

node *temp = newnode; temp->data = v; temp->prev = NULL; temp->next = NULL;

if (head == NULL & tail == NULL) { head = temp; tail = temp; }

else { node *p = head, *c = head; for (int i = 0; i < loc; i++) { p = c; c = c->next; }

tail->next = temp; temp->data = v; p->next = temp; temp->prev = p; c->prev = temp;

temp->next = c; }

LUCKY



Mon Tue Wed Thu Fri Sat Sun

Date

Insert after a given Value

void insert_after_value (int x, int v) {

node *temp = new node; temp->data = v; temp->next = NULL; temp->prev = NULL;

if (head == NULL || tail == NULL) { head = temp; tail = temp; }

else { node *p = head, *c = head;

while (c != NULL && c->data != x) {

p = c; c = c->next;

} if (c == NULL) {

cout << "value not found" << endl;

} else {

temp->next = c->next;

temp->prev = c;

if (c->next != NULL) {

c->next->prev = temp;

else

tail = temp;

c->next = temp;

}

}

}

}

node *p = head;

if (p->data == x) {

cout << "value not found" << endl;

return;

LUCKY



Mon Tue Wed Thu Fri Sat Sun

Date

Delete at head

```
void delete_at_head() {  
    node *q = head; head = head->next; head->prev = NULL;  
    q->next = NULL; delete q; q = NULL;  
}
```

Delete at tail

```
void delete_at_tail() {  
    node *q = tail; tail = tail->prev; tail->next = NULL;  
    q->prev = NULL; delete q; q = NULL;  
}
```

Delete at a specified location

```
void delete_at_loc(int i) {  
    node *pre = head, *curr = head;  
    for(int i = 0; i < i-1; i++) {  
        pre = curr; curr = curr->next;  
    }  
    pre->next = curr->next;  
    curr->next->prev = pre;  
    delete curr;  
    curr = NULL;  
}
```

Traversal / Display

void display()

```
node *p = head  
while (p != NULL) {  
    cout << p->data << " ";  
    p = p->next;  
}
```

void Reverse-Display()

```
node *q = tail;  
while (q != NULL) {  
    cout << q->data << " ";  
    q = q->prev;  
}
```

LUCKY



ADVANTAGES

- Bidirectional travel
- Insertion, deletion, & searching more efficient
- Predecessor & Successor of any element can be found quickly

DISADVANTAGES

- Consumes more memory due to extra pointer
- Larger pointer adjustment
- Consumes more time for basic operations because of that extra bookkeeping

void moveLastToFirst() {

if (head == NULL || head->next == NULL)

return;

Node * last = head;

while (last->next != NULL)

last = last->next;

last->prev->next = NULL;

last->prev = NULL;

last->next = head;

head->prev = last;

head = last; }

void reverseList() {

Node * current = head; Node * temp = NULL;

while (current != NULL) {

temp = current->prev;

current->prev = current->next;

current->next = temp;

current = current->prev;

}

if (temp != NULL)

head = temp->prev;

LUCKY



Circular Linked List

A singly or doubly linked list where the last Node's next points to the first node

Struct Node {

int data;

Node* next;

}

class clist {

private:

node * tail;

public:

clist() {

tail = NULL;

}

};

Add to head

void Add-to-head(int v) {

node * temp = new node; temp->data = v; temp->next = NULL;

if (tail == NULL) {

tail = temp; tail->next = temp;

}

else {

temp->next = tail->next;

tail->next = temp;

tail = temp (Insertion at tail)

}

}

LUCKY



Add at a specified location

```
void add_to_loc (int l, int v) {
```

```
    node *temp = new node;
```

```
    temp->data = v;
```

```
    temp->next = NULL;
```

```
    if (tail == NULL) {
```

```
        tail = temp; tail->next = temp;
```

```
    }
```

```
    else {
```

```
        node *pre = tail, *curr = tail;
```

```
        for (int i = 0; i <= l - 1; i++) {
```

```
            pre = curr; curr = curr->next;
```

```
            pre->next = temp; temp->next = curr;
```

```
        }
```

Delete at Start

```
void delete_at_start () {
```

```
    node *q = tail->next; tail->next = q->next; delete q; q = NULL;
```

```
}
```

Delete at end

```
void delete_at_end () {
```

```
    node *q = tail; node *temp = tail;
```

```
    while (temp->next != tail)
```

```
        temp = temp->next;
```

```
    tail = temp;
```

```
    tail->next = q->next;
```

```
}
```



Traversal / Display

```
void Display() {  
    node *p = tail->next;  
    do {  
        cout << p->data << " ";  
        p = p->next;  
    } while (p != tail->next);  
}
```

Advantages

- whole list can be traversed from any point.
- Fewer special cases
- Useful in implementation of other data structures

Disadvantages

- Finding end of list & loop control is harder